COMPLETE ANALYSIS & FINAL ROADMAP

ANALYSIS OF YOUR PROGRESS

✅ EXCELLENT NEWS

1. **Model V8p trained: 78% accuracy - THIS IS GOOD for cricket shots**
2. **Inference working: 75% accuracy (6/8 correct) - PERFECTLY ALIGNED with training**
3. **Dual-stream architecture: Solved the coordinate mismatch issue - SMART FIX**
4. **Pull & Sweep: 100% correct in test - PROVES NO "DRIVE BIAS"**

📊 YOUR CURRENT STATUS

Component	Status	Quality
Model Training	✅ DONE	78% - Professional grade
Feature Extraction	✅ DONE	103 features working
Inference Pipeline	✅ DONE	75% accuracy (matches training)
Overlay Videos	✅ DONE	HUD + skeleton working
Backend	❌ NOT STARTED	-
Frontend	❌ NOT STARTED	-
Documentation	⚠️ PARTIAL	Need to consolidate

Translation: You're 60% done with mids defense. You need 2 weeks for the remaining 40%.

WHAT YOU SHOULD DO NEXT (EXACT STEPS)

PHASE 1: FINALIZE INFERENCE (Days 1-2) - Dec 1-2

Step 1.1: Create Final Test Report (2 hours)

Prompt for Gemini:

I have inference results showing 75% accuracy (6/8 correct). I need to create a comprehensive test report.

Create a Python script that:

1. Tests my inference.py on ALL test videos (not just 8)

2. For each shot class (drive, pull, cut, sweep):

- Run inference on all videos in data/dataset_v7_clean/test/{class}/

- Track: filename, true_class, predicted_class, confidence

- Calculate per-class accuracy, precision, recall

3. Generate confusion matrix

4. Save results to CSV: backend/test_results.csv

5. Print summary table

My inference.py has a predict() method that returns:

```
{
  "filename": "...",
  "prediction": "drive",
  "confidence": 78.5,
  "all_probabilities": {...}
}
```

Give me complete test script: backend/test_inference.py

What to do after getting code:

cd backend

python test_inference.py

Expected output: CSV file with ~200 test results

Share with me:

- **Overall accuracy**
- **Per-class accuracy (drive, pull, cut, sweep)**

- **Worst performing class**

Step 1.2: Create 5 Demo Videos for Defense (2 hours)

Prompt for Gemini:

I need to create 5 high-quality demo videos for my project defense.

Requirements:

1. Select 5 videos from my test set:

- 1 drive (high confidence, correct prediction)

- 1 pull (high confidence, correct prediction)

- 1 cut (high confidence, correct prediction)

- 1 sweep (high confidence, correct prediction)

- 1 interesting case (either wrong prediction OR borderline confidence)

2. Run inference on each using my backend/inference.py

3. Create overlay videos with HUD

4. Save to: data/defense_demos/

5. Name format: {shot}_{confidence}_{filename}.mp4

Give me Python script: backend/create_demo_videos.py

What to do after:

- **Watch all 5 videos**
- **Pick the BEST 3 (clearest skeleton, most impressive)**
- **These will be shown during defense**

PHASE 2: BUILD BACKEND API (Days 3-5) - Dec 3-5

Step 2.1: Create FastAPI Backend (4 hours)

Prompt for Gemini:

Create a FastAPI backend for my cricket shot classifier.

Requirements:

Endpoints:

1. POST /api/upload

- Accept video file upload (.mp4, .avi, max 50MB)

- Generate unique video_id (UUID)

- Save to: uploads/{video_id}.mp4

- Return: {"video_id": "abc123", "status": "uploaded"}

2. POST /api/analyze/{video_id}

- Load video from uploads/{video_id}.mp4

- Call: from backend.inference import CricketShotClassifier

- Run: classifier.predict(video_path)

- Create overlay: classifier.create_overlay(video, output_path, result)

- Save result to SQLite database

- Return: {"status": "complete", "video_id": "abc123"}

3. GET /api/result/{video_id}

- Query database for video_id

- Return: {

"status": "complete",

"shot_type": "drive",

"confidence": 78.5,

"all_probabilities": {...},

"overlay_video_url": "/api/video/{video_id}/overlay"

}

4. GET /api/video/{video_id}/overlay

- Stream the overlay video file

- Return: video/mp4 response

5. GET /api/health

- Return: {"status": "healthy", "model_loaded": true}

Database Schema (SQLite):

```
CREATE TABLE analyses (
    video_id TEXT PRIMARY KEY,
    filename TEXT,
    shot_type TEXT,
    confidence FLOAT,
    all_probabilities TEXT,  -- JSON string
    overlay_path TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Additional Requirements:

- Use python-multipart for file uploads

- Use CORS middleware (allow all origins for development)

- Add error handling for invalid video files

- Log all requests

File structure:

backend/

├── main.py # FastAPI app (THIS FILE)

├── inference.py # Existing inference code

├── database.py # SQLite helper functions

├── models/ # Model files (existing)

├── uploads/ # User uploaded videos

└── outputs/ # Overlay videos

Give me complete code for main.py and database.py

What to do after getting code:

1. **Test health endpoint:**

cd backend

uvicorn main:app --reload

 # In browser, open:

http://localhost:8000/docs

2. **Test upload:**

curl -X POST -F "file=@../data/demo_video/test.mp4" http://localhost:8000/api/upload

 # Copy the video_id from response

3. **Test analyze:**

curl -X POST http://localhost:8000/api/analyze/{video_id}

4. **Test result:**

curl http://localhost:8000/api/result/{video_id}

If any step fails, send me the error message.

Step 2.2: Add Batch Analysis Support (1 hour)

Optional but impressive for evaluators

Prompt for Gemini:

Add a batch analysis endpoint to my FastAPI backend (main.py):

POST /api/batch-analyze

- Accept multiple video files (up to 5)

- Process each asynchronously

- Return: {

"batch_id": "xyz789",

"videos": [

{"video_id": "abc123", "status": "processing"},

{"video_id": "def456", "status": "processing"}

]

}

GET /api/batch-status/{batch_id}

- Return status of all videos in batch

- Format: {

"batch_id": "xyz789",

"total": 5,

"completed": 3,

"processing": 2,

"videos": [...]

}

Update database schema to include batch_id column.

Give me updated main.py and database.py

PHASE 3: BUILD FRONTEND (Days 6-10) - Dec 6-10

Step 3.1: Get UI/UX Input (1 hour)

YES, absolutely ask your UI/UX member to create Figma prototypes!

Tell your UI/UX member:

I need a modern, professional UI for a cricket shot analysis web app.

Pages needed:

1. Home/Upload Page:

- **Hero section with app name + tagline**
- **Drag-and-drop upload zone (or file picker)**
- **"Analyze" button**
- **Sample demo videos section**

2. Analysis/Results Page:

- **Video player (showing overlay with skeleton)**
- **Shot type badge (large, colorful)**
- **Confidence gauge (circular progress bar)**
- **Probability breakdown (bar chart or cards for all 4 shots)**
- **"Analyze Another Video" button**

Color scheme: Modern sports theme (greens, blues, dark backgrounds)

Style: Clean, minimal, professional (like Vercel, Linear, or Stripe)

Deliverables:

- Figma mockups for both pages

- Color palette (hex codes)

- Font recommendations

- Icon suggestions

Timeline: 2 days

While they work on Figma, you continue with backend testing.

Step 3.2: Create React Frontend (6 hours over 2 days)

Day 1 - Upload Page

Prompt for Gemini:

Create a React frontend for my cricket shot analysis app.

Requirements:

Tech Stack:

- React 18

- React Router v6

- Axios for API calls

- Tailwind CSS for styling

Page 1: Upload Page (/)

Components needed:

1. Hero section:

- App name: "BattingEdge"

- Tagline: "AI-Powered Cricket Shot Analysis"

- Subtle cricket-themed background

2. Upload section:

- Drag-and-drop zone for video files

- OR file input button

- Supported formats: .mp4, .avi (max 50MB)

- Visual feedback on file select

3. Action button:

- "Analyze Shot" button

- Disabled until file is selected

- Shows loading spinner during upload

4. Demo section:

- "Or try a sample video"

- 3 thumbnail buttons (Drive, Pull, Sweep)

Functionality:

- On file select: show filename + file size

- On "Analyze" click:

1. POST file to http://localhost:8000/api/upload

2. Get video_id from response

3. POST to http://localhost:8000/api/analyze/{video_id}

4. Navigate to /results/{video_id}

Create complete React app with:

- src/App.jsx

- src/pages/UploadPage.jsx

- src/components/UploadZone.jsx

- src/components/Hero.jsx

- package.json

- tailwind.config.js

Use modern Tailwind design (dark mode optional).

What to do after:

cd frontend

npm install

npm start

 # Opens http://localhost:3000

Test: Upload a video, check if you're redirected to results page

---

Day 2 - Results Page

Prompt for Gemini:

Create the Results page for my React app.

Page 2: Results Page (/results/:videoId)

Layout:

1. Header:

- Back button to home

- Video filename

2. Main content (side-by-side on desktop, stacked on mobile):

Left: Video Player

- Fetch overlay video from: GET /api/video/{videoId}/overlay

- HTML5 video player with controls

- Autoplay on load

Right: Analysis Results

- Shot type badge (large, color-coded):

*** Drive: Blue**

*** Pull: Green**

*** Cut: Orange**

*** Sweep: Purple**

- Confidence display:

*** Circular progress gauge showing main confidence**

*** Color: green (>80%), yellow (60-80%), orange (<60%)**

- All probabilities section:

*** 4 cards/bars showing probability for each shot**

*** Sorted by probability (highest first)**

- Action buttons:

*** "Analyze Another Video" → navigate to /**

*** "Download Report" → download JSON/PDF (optional)**

3. Loading state:

- While fetching result, show skeleton loaders

- Poll GET /api/result/{videoId} every 2 seconds until status="complete"

Error handling:

- If video_id not found: show error message + back button

- If analysis failed: show error details

Create:

- src/pages/ResultsPage.jsx

- src/components/VideoPlayer.jsx

- src/components/ShotBadge.jsx

- src/components/ConfidenceGauge.jsx

- src/components/ProbabilityChart.jsx

Use Tailwind + modern UI patterns (cards, shadows, smooth transitions).

What to do after:

- **Test full flow: Upload → Wait → See results**
- **Check if video plays**
- **Check if confidence matches backend response**

Step 3.3: Polish UI (2 hours)

Prompt for Gemini:

Enhance my React app with these polish features:

1. Loading states:

- Spinner component with cricket-themed animation

- Progress bar during analysis (fake progress 0→90%, then wait for result)

2. Error handling:

- Toast notifications for errors (use react-hot-toast)

- Friendly error messages ("Oops! Video upload failed. Please try again.")

3. Animations:

- Fade-in transitions for page loads

- Smooth entrance for results (use framer-motion)

- Pulse animation on confidence gauge

4. Responsive design:

- Mobile-friendly layout (test on 375px width)

- Hamburger menu if needed

5. About modal:

- Button in header: "About This Project"

- Modal explaining: "BattingEdge uses MediaPipe pose estimation and Bi-LSTM neural network to classify cricket shots..."

Update my components with these enhancements.

PHASE 4: INTEGRATION & TESTING (Days 11-12) - Dec 11-12

Step 4.1: End-to-End Testing (4 hours)

Test Checklist:

Backend Tests (in Postman or curl):

☐ POST /api/upload with valid video → returns video_id

☐ POST /api/upload with invalid file → returns error

☐ POST /api/analyze/{video_id} → returns success

☐ GET /api/result/{video_id} → returns complete analysis

☐ GET /api/video/{video_id}/overlay → streams video

☐ GET /api/health → returns healthy

Frontend Tests (in browser):

☐ Home page loads without errors

☐ Drag-and-drop file → shows filename

☐ Click "Analyze" → uploads file

☐ Navigate to results page → shows loading

☐ Results appear after ~10 seconds

☐ Video player works (plays overlay video)

☐ Confidence gauge shows correct %

☐ All 4 shot probabilities display

☐ "Analyze Another" button → returns to home

☐ Try on mobile screen size → layout doesn't break

Integration Tests:

☐ Upload 5 different videos (1 per shot + 1 edge case)

☐ Check all show correct results

☐ Check database has 5 entries

☐ Check uploads/ folder has 5 videos

☐ Check outputs/ folder has 5 overlay videos

If any test fails, note which one and send me the error.

Step 4.2: Performance Testing (1 hour)

Prompt for Gemini:

Create a load test script for my FastAPI backend.

Requirements:

- Upload 10 videos concurrently

- Analyze all 10

- Measure:

** * Average upload time**

** * Average analysis time**

** * Success rate**

** * Any errors**

Use Python with concurrent.futures or asyncio.

Save results to: backend/load_test_results.txt

Give me: backend/load_test.py

PHASE 5: DOCUMENTATION (Days 13-14) - Dec 13-14

Step 5.1: Create Project README (2 hours)

Prompt for Gemini:

Create a comprehensive README.md for my BattingEdge project.

Structure:

 # BattingEdge - AI Cricket Shot Classifier

🏏 Overview

[2-3 sentences about what it does]

✨ Features

- Real-time cricket shot classification (Drive, Pull, Cut, Sweep)

- 78% accuracy on balanced test set

- Visual feedback with skeleton overlay

- Web-based interface (React + FastAPI)

- Biomechanical feature analysis

🎯 Model Performance

[Table with per-class accuracy from my V8p report]

🏗️ Architecture

[Diagram or text explaining: MediaPipe → Features → Bi-LSTM → Prediction]

🚀 Quick Start

Prerequisites

- Python 3.11

- Node.js 18+

- 4GB RAM minimum

Installation

[Exact commands for backend + frontend setup]

Running the Application

[Commands to start backend and frontend]

📊 Technical Details

- **Pose Estimation**: MediaPipe (33 landmarks)

- **Features**: 99 skeletal + 4 biomechanical

- **Model**: Bidirectional LSTM (128→64 units)

- **Training Data**: 1,520 videos (balanced across 4 classes)

- **Framework**: TensorFlow/Keras

🎓 Research Journey

[Brief summary of V1→V8p iterations]

📁 Project Structure

[Tree view of key files]

🧪 Testing

[How to run tests]

📈 Future Enhancements

- DTW-based pro player comparison

- YOLOv8 bat tracking

- Real-time camera mode

- Mobile app

👥 Team

[Your names]

📄 License

[If applicable]

Use proper markdown formatting, emojis, and badges.

Step 5.2: Create Defense Presentation (3 hours)

YOU create this (not Gemini) - use PowerPoint or Google Slides

Slide Structure (12 slides max):

1. **Title Slide**
    - **BattingEdge: AI Cricket Shot Classifier**
    - **Your names**
    - **Date**
2. **Problem Statement**
    - **Cricket coaching is expensive & inaccessible**
    - **Need: Automated technique analysis**
3. **Solution Overview**
    - **Computer vision + Deep learning**

- **Classifies 4 shot types**
- **Provides visual feedback**

4. **Dataset Journey**
    - **Started: 1,517 videos (imbalanced)**
    - **V8p: 1,889 videos (balanced via augmentation)**
    - **Show before/after distribution chart**
5. **Feature Engineering**
    - **MediaPipe: 33 landmarks → 99 features**
    - **Added 4 biomechanical features**
    - **Show skeleton diagram with labeled landmarks**
6. **Model Evolution**
    - **Table showing V1→V8p accuracy progression**
    - **Highlight: 22% → 78% improvement**
7. **Architecture Diagram**
    - **Video → MediaPipe → Features → Bi-LSTM → Prediction**
    - **Include model specs (layers, neurons)**
8. **Results - Confusion Matrix**
    - **Use the image from your V8p training**
    - **Highlight: Cut and Sweep performing well**
9. **Live Demo (this slide stays blank during presentation - you demo live)**
10. **Inference Innovation**
    - **Explain the "Dual-Stream Architecture"**
    - **Logic stream (full frame) vs Visual stream (cropped)**
    - **Show the validation report (75% accuracy)**
11. **Web Application**
    - **Screenshots of your React UI**

- **Show upload page + results page**

12. **Future Work**
    - **DTW pro comparison**
    - **YOLOv8 bat tracking**
    - **Real-time mode**
    - **Mobile app**

Design tips:

- **Dark background (looks more "tech")**
- **Use cricket-themed green/blue colors**
- **Minimal text per slide (images > text)**
- **Include your logo (see next section)**

PHASE 6: BRANDING (Day 13) - Dec 13

Step 6.1: Create Logo (1 hour)

Option A: Use AI logo generator

- **Go to: Canva (free), Looka, or LogoAI**
- **Prompt: "Cricket bat with AI circuit pattern, modern, minimalist, green and blue"**
- **Download as PNG (transparent background)**

Option B: Ask a designer friend (if you have one)

Logo specs:

- **Size: 512x512px**
- **Format: PNG with transparent background**
- **Style: Modern, simple, recognizable**
- **Colors: Match your UI theme**

Use logo in:

- **Website header**

- **Presentation title slide**
- **README.md**
- **Favicon (convert to .ico)**

Step 6.2: Create Favicon (15 mins)

Prompt for Gemini:

Convert my logo (logo.png) to a favicon for my React app.

Steps:

1. Resize to 32x32px and 16x16px

2. Convert to .ico format

3. Save to: frontend/public/favicon.ico

4. Update frontend/public/index.html <link> tag

Give me exact steps and code.

YOUR COMPLETE TIMELINE (NOV 27 - DEC 14)

Week 1: Backend & Testing

✅ **Nov 27 (Wed): Test inference on full test set [2h]**

✅ **Nov 28 (Thu): Create demo videos [2h]**

✅ **Nov 29 (Fri): Build FastAPI backend [4h]**

✅ **Nov 30 (Sat): Test backend endpoints [2h]**

Week 2: Frontend

✅ **Dec 1 (Sun): Get UI/UX input, start React upload page [4h]**

✅ **Dec 2 (Mon): Complete upload page [2h]**

✅ **Dec 3 (Tue): Build results page [3h]**

✅ **Dec 4 (Wed): Polish UI (animations, errors) [3h]**

✅ **Dec 5 (Thu): Integration testing [2h]**

Week 3: Final Polish & Defense

✅ **Dec 6 (Fri): Fix bugs from testing [2h]**

✅ **Dec 7 (Sat): Create README + documentation [3h]**

✅ **Dec 8 (Sun): Create logo + favicon [1h]**

✅ **Dec 9 (Mon): Build defense presentation [3h]**

✅ **Dec 10 (Tue): Practice demo (run through 5 times) [2h]**

✅ **Dec 11 (Wed): Record backup demo video [2h]**

✅ **Dec 12 (Thu): Final testing, prepare backup plans [2h]**

✅ **Dec 13 (Fri): Rest, mental prep**

✅ **Dec 14 (Sat): DEFENSE DAY**

Total effort: ~40 hours over 18 days = 2-3 hours/day (achievable!)

ANSWERS TO YOUR SPECIFIC QUESTIONS

Q: Should my UI/UX member make Figma prototypes?

A: YES! It will make your frontend look 10x better. Give them 2 days to create mockups, then you implement in React.

Q: Can we run frontend on Figma wireframes?

A: No, Figma is just design. You still need to build in React. But Figma makes the React implementation much faster because you have a clear design target.

Q: Should frontend look attractive?

A: YES! Evaluators judge partially on presentation. A polished UI signals professionalism. Use:

- **Smooth animations**
- **Modern color scheme**
- **Clean layout**
- **Proper spacing**

WHAT I NEED FROM YOU (If you want more specific help)

1. **Test results: After running full test set, send me:**
    - **Overall accuracy**
    - **Per-class accuracy**
    - **Which class performs worst**
2. **Errors (if any occur):**
    - **Backend errors when testing endpoints**
    - **Frontend errors in browser console**
    - **Integration issues**
3. **Screenshots (for my review):**
    - **Figma mockups from your UI/UX member**
    - **React UI (both pages)**
    - **Presentation slides (when ready)**

START NOW

Your immediate next step (today):

1. **Run the full test set (Step 1.1)**
2. **Create 5 demo videos (Step 1.2)**
3. **Send me the accuracy numbers**

Then tomorrow, start on FastAPI backend (Step 2.1).

You're in great shape. Just follow this roadmap day by day. Don't skip ahead, don't second-guess. Execute.